

Integrating Machine Learning and Memory Forensics for Enhancing Cybersecurity in IoT-Enabled Energy Systems

^aSonika Malik, ^bVedaang Chopra, ^cMonica Bhutani

^aAssistant Professor, MSIT, Department of IT, New Delhi, India

^bStudent, MSIT, Department of IT, New Delhi

^cAssociate Professor, BVCOE, Department of ECE, New Delhi, India

sonika.malik@msit.in, vedaangchopra1009@gmail.com, monica.bhutani@bharativedyapeeth.edu

Abstract

Malwares have constantly threatened every individual and organization. Even though antivirus and endpoint security tools exist, the number of attacks and damage costs keep increasing exponentially. Systems are compromised; organizations must pay hefty amounts to release their data. Daily private users are victims of such Malware Attacks. This study introduces an innovative architecture-independent malware detection system designed to address the diverse threats posed by malware, which are developed in various high-level programming languages like C, Python, and Java and perform specific malicious functions. Recognizing that malware, such as ransomware, Trojans, and viruses, are ultimately translated into low-level machine code to execute their intended actions, our system leverages this commonality to identify similarities in machine code patterns across different malware families. By focusing on the fundamental aspect that malware within the same family, such as ransomware, share a core functionality—hence similar machine code signatures—our approach offers a robust mechanism for detecting a wide range of malware types. This abstract underscores our contribution to malware detection research, presenting a system capable of identifying malicious software through its machine code, independent of the architecture, thereby enhancing the effectiveness of cybersecurity measures.

Keywords: Artificial Intelligence, cyber Security, Memory Forensics, Dynamic Analysis, YARA Rules, Machine Learning

1. Introduction

Endpoint Security Mechanic systems serve as protective measures for various devices such as laptops, desktops, and smartphones. They are designed to shield these devices from all forms of attacks, whether they originate from within the device or over a network. Examples of such endpoint security measures for personal devices include antivirus software and firewall protections[1]. Malware, short for "malicious software," collectively refers to any malevolent program or code that poses a threat to computer systems[2][3]. Some common types of malware are [4]:

- A virus is a type of malware that latches onto another application and, once activated—often accidentally by

the user—duplicates itself by altering other software on the computer and integrating its malicious code.

- Worms are akin to viruses in that they are self-replicating malware designed to spread across networks to other devices, typically resulting in the destruction of data and files
- A Trojan, or Trojan horse, stands as one of the more perilous forms of malware. It masquerades as legitimate software to deceive users. Once installed, it provides hackers unauthorized entry to the system, allowing them to pilfer financial data or deploy further malicious threats such as viruses and ransomware.
- Ransomware is a malicious software variant that denies access to your device or encrypts your files, subsequently demanding a ransom for their release. Recognized for its fast monetary gains through anonymous cryptocurrency transactions, ransomware's programming is readily accessible on illicit online markets, making its mitigation challenging.
- A rootkit offers cyber attackers administrator-level access to a compromised system, ingeniously concealing its presence from both the user and system defenses.
- Keyloggers stealthily record every keystroke made on an infected device, typically to harvest and transmit confidential data like usernames, passwords, and financial details to cybercriminals.
- Exploits leverage flaws and security gaps in systems to grant the attacker control. Often associated with malevolent advertising, they exploit reputable websites to unwittingly serve harmful content from malicious sources. This content attempts auto-installation on visitors' devices without the need for interaction, merely requiring the unlucky timing of accessing a compromised site.

Antiviruses have deployed various techniques to classify a file as malicious or not [2][3] [5]:

- a. Signature-based Detection: In this detection technique, the traditional method of matching any program to its signature (Example: hash) is

employed. It is a premature technique and can be easily bypassed as a slight change in the program can immediately change its signature. For this technique to work seamlessly, it requires a large database of known malicious files.

- b. Heuristic-based detection employs algorithms to match known virus signatures with possible threats, making it a widely used detection method. Antivirus software equipped with heuristic detection can identify not only existing viruses but also new, previously undiscovered ones. However, this method can sometimes lead to false positives, where the antivirus incorrectly flags a clean file as infected.
- c. Behavioral-based detection: This detection method monitors the malicious program using the dynamic malware analysis technique. After analyzing the changes made to the system by the process, it classifies whether the file is malware or not. Some of the characteristics of the process that are monitored are as follows:
 - Creation of files
 - Deletion of files
 - Memory consumed
- d. Sandbox detection: This is a type of detection method in which antivirus software runs programs in a virtual environment and records the actions it performs to identify whether the programs are malicious or not. If the program is found safe, it is then executed in the real environment.

2. Background Information

The following concepts are needed to understand this research [6][7]:

2.1. Malware Analysis

The task of analyzing malwares and identifying what unique signatures a malware has is known as Malware Analysis. These signatures can include strings in the malware code, author name, server to which it connects, etc. It is categorized into two parts [8]:

Static Analysis

This is a process whereby those properties of malware that can be identified without executing it are analyzed. Some examples of such properties are

- i. File type
- ii. PE Header
- iii. Packer used
- iv. Strings etc.

Dynamic Analysis

This is a process where we try analyzing those properties of malware that can be identified by executing it in an environment. Some examples of such properties are

- v. Resource consumption
- vi. Network traffic generated
- vii. Files Dropped/Created, etc.

a. Memory Forensics

Digital forensics, a branch of forensic science, works on encompassing the recovery and investigation of material found in digital devices, often with computer crime [9].

Memory Forensics is a digital forensics field involving the recovery and investigation of a digital device's memory [10].

Volatility stands as a utility designed for Memory Forensics tasks. Developed under the GNU General Public License, the Volatility Framework is an openly available set of tools, crafted in Python, for retrieving digital artifacts from volatile memory (RAM) samples. These retrieval methods operate entirely separately from the system under analysis, yet they allow for an inspection of the system's operational state at runtime. Aimed at acquainting individuals with the methodologies and intricacies involved in extracting digital artifacts from RAM samples, the framework also serves as a foundation for ongoing research in this dynamic field.

2.2. YARA

YARA functions as a malware identification tool through the creation of "rules" that search for specific attributes within files[12]. These rules are formulated using either textual or binary patterns. With its capacity to utilize regex patterns, YARA facilitates the identification of particular sequences in files that could signify malicious intent. Its versatile rule system can recognize various forms of malware signatures, incorporating hex patterns, straightforward text sequences, wildcards, strings insensitive to case, and unique operators[13].

2.3. Machine Learning

This approach involves teaching machines to understand and learn from previous data or experiences. For instance, humans learn to navigate crossing the road through experience by observing and making decisions based on their surroundings.

Machine learning can be divided into three main categories:

- Supervised Learning,
- Unsupervised Learning,
- Reinforcement Learning.

In Supervised Learning, every input is matched with an explicit output, making the learning outcome very specific. There are two subdivisions within Supervised Learning: Regression and Classification. In our research, we tackled a problem through Supervised Learning by focusing on Classification. The classification techniques we utilized are detailed below.

Naive Bayes classifier-

The Naive Bayes classifier is effective for situations involving the classification of multiple categories. It employs the principles of Bayes' theorem for making predictions. Imagine a scenario with a data point x and three distinct categories: A, B, and C. Naive Bayes attempts to determine the likelihood of x belonging to

either category A, B, or C by computing its probability for each class.

$$P(A/B) = (P(B/A) \cdot P(A))/P(B) \quad (1)$$

This method is termed "Naive" due to its underlying assumption that all features within the data are mutually independent.

$$P(f_1, f_2, f_3, \dots, f_n) = P(f_1) \cdot P(f_2) \cdot \dots \cdot P(f_n) \quad (2)$$

Naive Bayes Classifier is further divided into two sub-types: the Gaussian Naive Bayes Classifier and the Multinomial Naive Bayes Classifier. The Gaussian variant is preferred for datasets with continuous data values, leveraging the Gaussian probability distribution function, where values are normally distributed around the mean, tapering off as they move away from it.

Decision Tree-

An ensemble approach involves developing a decision tree derived from decisions made based on the dataset. Constructing a decision tree entails certain steps: (i) identifying a feature to divide the dataset, (ii) choosing a technique to segment the data based on the selected feature. The internal mechanism of this method includes:

- (i) Identifying or choosing a node.
- (ii) If the node is homogeneous, return the single class.
- (iii) If there are no remaining features to split on and the node is heterogeneous, return the class with the majority.
- (iv) Otherwise, identify the optimal feature for division. Apply the process recursively on this division. Proceed to step (ii).

KNN

In a situation where the goal is to classify a test data point based on evaluating all features simultaneously, the KNN (K-Nearest Neighbors) algorithm comes into play, as illustrated in Fig 1. The method involves determining the class of the test point by examining the classes of its nearest neighbors.

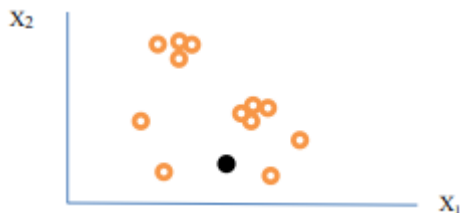


Fig. 1 KNN Algorithm

For classification, a predetermined number of nearest points (e.g., 1, 3, 5, 7, etc.) are considered, and the test point is assigned to the class that is most common among these neighbors. To identify these nearest points, their distance from the test point is calculated using various metrics, such as (a) Manhattan distance, (b) Euclidean distance, and (c) Minkowski distance.

Random forest

The Random Forest classifier enhances the decision tree model by employing a combination of multiple decision trees to make more accurate predictions on test data. This approach effectively addresses the issue of overfitting associated with single decision trees by aggregating the outcomes of numerous trees. Each tree in the forest may produce different results due to the random selection of data and features used in their construction. To introduce randomness and diversity among the trees, two key strategies are employed:

- **Bagging Classifier:** This method involves selecting a random subset of 'k' data points from the total 'm' available for each of the 'n' trees, resulting in 'n*k' subsets. Since the selection is random and allows for replacement, the trees generated are often referred to as bag trees.
- **Feature Selection:** During the training phase, this technique randomly selects a set of features for each tree, ensuring that each tree considers a different subset of features. This selection is done without replacement, further contributing to the variability and robustness of the model.

SVM Classifier

Support Vector Machines (SVM) are a robust set of algorithms that address some of the constraints found in logistic regression. While logistic regression relies on the sigmoid function to model probabilities, leading to predictions that can hover near the decision boundary (around 0.5), this can sometimes result in ambiguous classifications. SVM, however, operates differently. It does incorporate principles from logistic regression but significantly enhances the decision-making process by ensuring that the predicted values are more distinctly separated. Instead of working within a narrow range like (-1, 1) with probabilities, SVM aims to maximize the margin between different classes, thus providing a clearer, more definitive classification boundary.

SVM does this by transforming the input data space into a higher dimension where a hyperplane can effectively separate the classes. The decision function in SVM thus doesn't confine itself to the output range of the sigmoid function, allowing for a more decisive separation between classes.

As for the cost function of logistic regression, it's typically represented as:

$$E(h(x)) = \sum (-y_i \log(h(x)) - (1-y_i) \log(1-h(x))) \quad (3)$$

This cost function changes to the following equation in SVM –

$$(\theta) = C \sum [y_{(i)} \text{cost}1(\theta T_{x(i)}) + (1-y_{(i)}) \text{cost}0(\theta T_{x(i)})] + 0.5 \sum (\theta_i)^2 \quad (4)$$

Gradient Boosting Algorithm

It continuously creates models that are nothing but improvisation on the previous version. It essentially keeps strengthening the model until a model strong enough for correct prediction is generated. This is done by building a model from the training data and then creating a second model that attempts to correct the errors from the first model. The addition of improved models terminates when either of the two outcomes is achieved – (i) the prediction is perfected or (ii) the upper limit for adding models is reached.

3. Related Work

Various techniques have been used by researchers to analyze and detect malware. In order to detect malware, several researchers used static analysis. Machine learning was used by Hashemi and Hamzeh [16] to detect malware after analyzing executable files to extract the unique opcode features. According to Salehi et al. [17], the most frequent API calls were selected to learn the classifier. Cheng et al. [18] extracted native APIs from shellcode using WinDbg and classified the malware using SVM. Based on static analysis, Sun et al. [19] identified malicious PE files with 32-bit and 64-bit opcode sequences. In order to distinguish different types of malicious PE files, the frequency of each opcode sequence is calculated. Data sets from Kaggle Microsoft Malware Classification Challenge (BIG 2015) and vx-heaven were analyzed. By applying the Adaboost classifier, the highest accuracy equaled 98.57%. A gradient-based attack by Kolosnjaji et al. [20] was also developed to evade malware detection by modifying a few bytes at the end of each malware file. Furthermore, dynamic analysis has been used by many authors in the detection and classification of malware. An in-depth evaluation of the techniques and tools that have been used in dynamic analysis has been published by Egele et al. in

[21]. Zeus malware was classified by Mohaisen et al. [22] using the registry file system and network features extracted from the registry. This led to the introduction of a behavior-based automated malware analysis and labeling system (AMAL) by Mohaisen et al. [23]. File system, registry, and network activity were studied to identify malware. A technique introduced by Liang et al. in [24] uses API dependency chains to calculate the similarity between malware variants. Galal et al. [25] similarly captured API call information and its parameters. Sequences were created for API calls with mutual purposes. The API hooking approach was also used by Fan et al. [26] in order to trace hidden API calls as well as native APIs. A similar sequence was aligned using a multiple sequence alignment algorithm (MSA) by Ki et al. [27]. According to Dai et al. [28], a memory dump can be used to categorize malware. An image in grayscale is created by extracting files from the memory dump. A fixed size is then applied to the image. A feature extraction algorithm was applied to the images to categorize malware based on API call features. A random forest was used to achieve the highest accuracy of 93.9%. In spite of the successful results achieved by the above methods, no other work attempts to integrate dynamic analysis with memory analysis to overcome their limitations. Some recent works in the same domain have occurred, advancing the current techniques [29][30].

4. Proposed Framework

The Project flow can be classified into four separate phases as shown in Fig. 2.

1. Lab Setup
2. Sample Generation
3. Memory Analysis

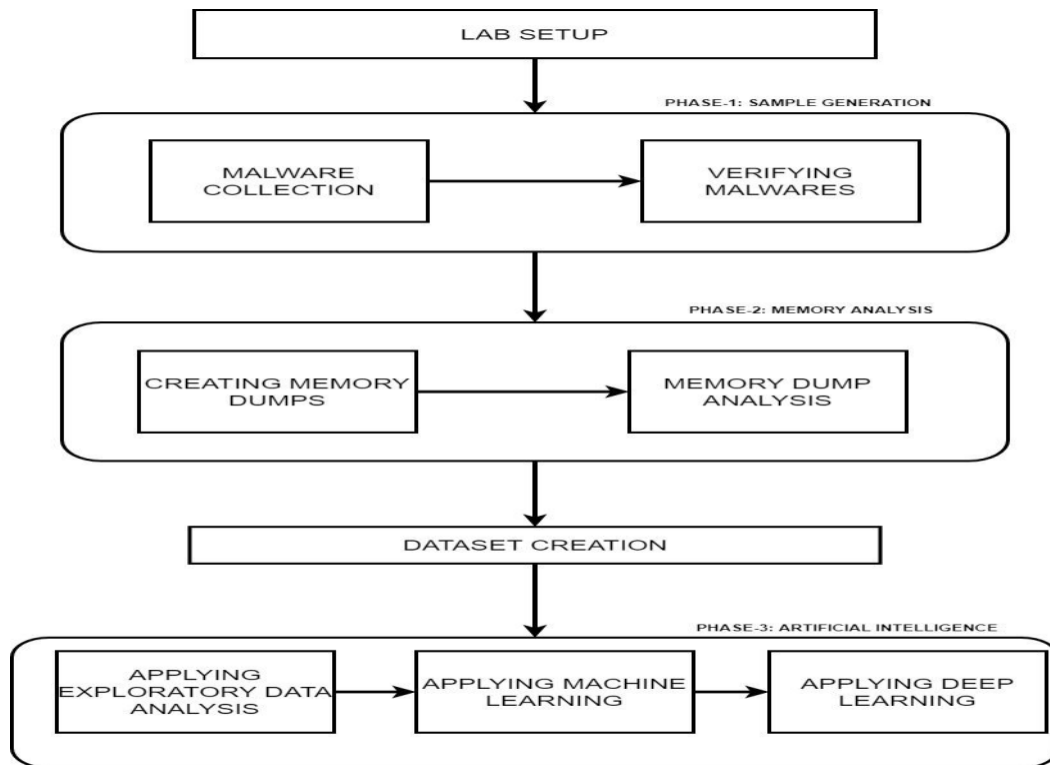


Fig. 2. Flow of Code of Project

4.1. Lab Setup

To analyze malware, there is a need to set up a complete LAB, including hypervisors, malware analysis tools, forensics tools, etc.

The following Labs were created: -

1. Malware Analysis Lab Setup
 - a. VMware machine with Windows 10 (trial)
 - b. Installation of FLARE-VM over Windows OS
2. Memory Forensics Lab Setup
 - a. VMware machine with UBUNTU 18.04
 - b. Installation of CUCKOO Sandbox
 - c. Installation of Virtual box Ubuntu Server with Inetsim
 - d. Installation of Virtual box machine with Windows 10 inside VM machine

4.2. Sample Generation

To collect live malwares, many public repository websites were used. Some of the repositories were: -

1. Any. apprun
2. theZOO
3. <https://github.com/Endermanch/MalwareDatabase>
4. malware-samples@fabrimgic

Even though the samples were collected, each of the samples still needed to be verified by running it in a sandboxed environment. For the initial analysis and dataset

creation process, only RANSOMWARES were considered. The virtual machine details are a guest operating system such as Windows 10 x64 and build version 18362. (Windows Developer trial)

The list of ransomwares used is as follows:

1. Annabelle.exe
2. Blue Howl.exe
3. Ceber.exe
4. Cerber1.exe(another version)
5. GandCrab.exe
6. Jigsaw1.exe
7. Jigsaw2.exe(another version)
8. TeslaCrypt1.exe
9. Vipasana1.exe
10. ViraLock.exe
11. WannaCry1.exe
12. WannaCry2.exe(another version)

4.3. Memory Analysis

This Memory Analysis phase consists of two parts:

- a. Creating Memory Dumps
- b. Analyzing Memory Dumps

Creating Memory Dumps: These techniques can be used to create a memory dump of a malware-executed sandbox:

- Use of Cuckoo Sandbox Automated Memory Dump feature.

- Use of VirtualBox command line feature.
COMMAND TO USE
VBoxManage debugvm <Virtual Machine name> dumpvmcore --filename=<filename>.elf

Analyzing Memory Dumps:

To analyze memory dumps, the use of Volatility (memory forensics tool) is required. It is used to extract all crucial information about the systems. In our project, the following steps and commands were used to analyze memory [16][17].

Steps:

1. Create the .elf into a .raw file for proper analysis using the imagecopy command. (elf files have an

issue. See http://wiki.yobi.be/wiki/RAM_analysis for further details.)

2. Identify the build profile of the memory dump using the imageinfo command.
3. Identify all the processes running in the system using the pslist command.
4. Identify all the hidden processes by running the psscan command and comparing the results with the pslist command output.
5. With your Yara rules collection, run the yarascan command to identify malicious memory samples.
6. Use this result for analysis.

Repeat the following steps for other memory dumps.

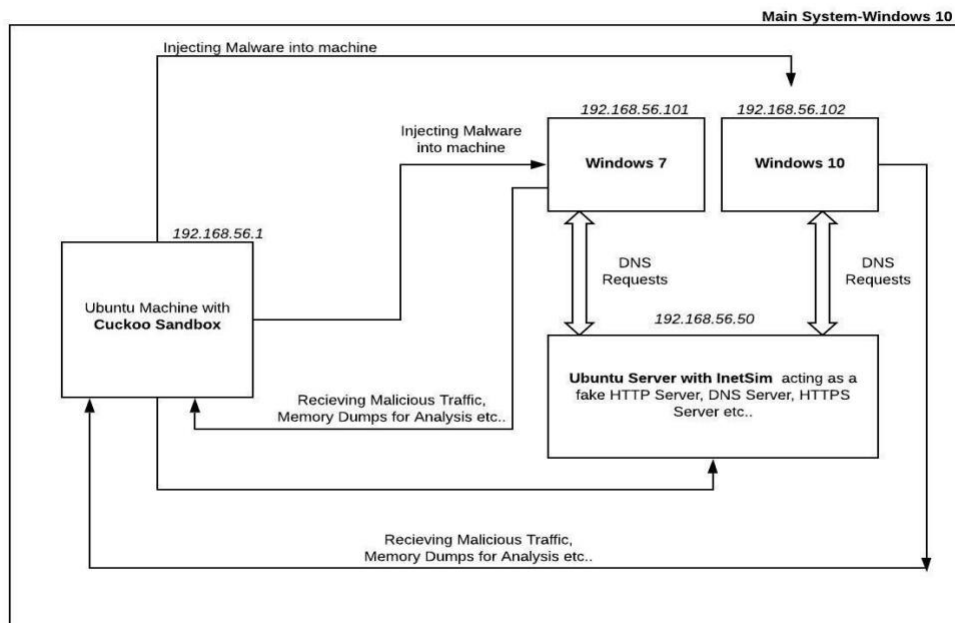


Fig. 3. Memory Forensics Lab Setup

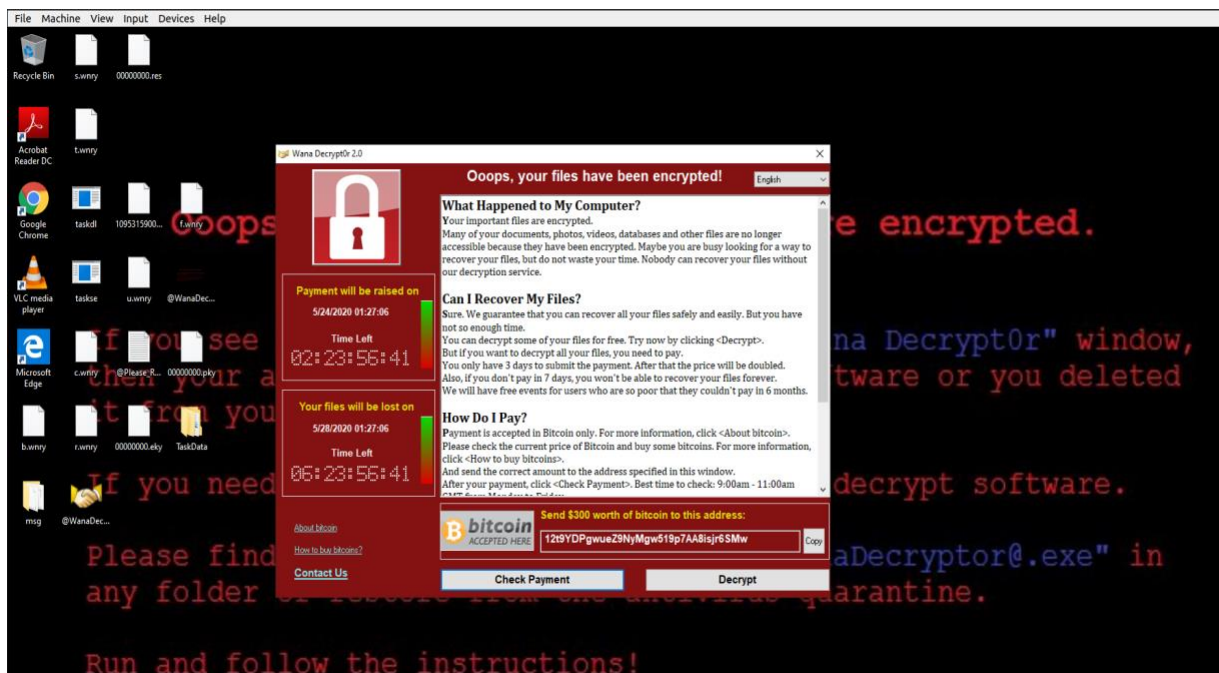


Fig. 4 Example of WannaCry Ransomware Executing in Sandbox (Live Malware Running)

```
cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$ vol.py -f WannaCry2.raw imageinfo
Volatility Foundation Volatility Framework 2.6.1
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win10x64_18362
                           AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
                           AS Layer2 : FileAddressSpace (/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis/WannaCry2.raw)
                           PAE type : No PAE
                           DTB : 0x1aa002L
                           KDBG : 0xf8053b8265e0L
      Number of Processors : 4
      Image Type (Service Pack) : 0
      KPCR for CPU 0 : 0xfffff8053855b000L
      KPCR for CPU 1 : 0xfffff8180b5fc0000L
      KPCR for CPU 2 : 0xfffff8180b6021000L
      KPCR for CPU 3 : 0xfffff8180b5be3000L
      KUSER_SHARED_DATA : 0xfffff78000000000L
      Image date and time : 2020-05-10 09:31:13 UTC+0000
      Image local date and time : 2020-05-10 15:01:13 +0530
cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$
```

Fig. 6. Identifying the Build profile of the Memory Dump.

```

Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x00879f28 74 61 73 6b 73 65 2e 65 78 65 00 00 40 57 61 6e taskse.exe..@Wan
0x00879f38 61 44 65 63 72 79 70 74 6f 72 40 2e 65 78 65 00 aDecryptor@.exe.
0x00879f48 74 61 73 6b 73 63 68 65 2e 65 78 65 00 00 00 tasksche.exe....
0x00879f58 25 00 73 00 5c 00 25 00 73 00 5c 00 25 00 73 00 %.s.\.s.\.s.\.s.
0x00879f68 00 00 00 00 25 00 73 00 5c 00 2a 00 2e 00 2a 00 ....%.s.\.*....*.
0x00879f78 00 00 00 00 40 57 61 6e 61 44 65 63 72 79 70 74 ....@WannaDecrypt
0x00879f88 6f 72 40 2e 65 78 65 2e 6c 6e 6b 00 5c 00 00 00 or.exe.lnk.\...
0x00879f98 40 65 63 68 6f 20 6f 66 66 0d 0a 65 63 68 6f 20 @echo.off..echo.
0x00879fa8 53 45 54 20 6f 77 20 3d 20 57 53 63 72 69 70 74 SET.ow.=.WScript
0x00879fb8 2e 43 72 65 61 74 65 4f 62 6a 65 63 74 28 22 57 .CreateObject("W
0x00879fc8 53 63 72 69 70 74 2e 53 68 65 6c 6c 22 29 3e 20 Script.Shell")>.
0x00879fd8 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 53 45 54 20 m.vbs..echo.SET.
0x00879fe8 6f 6d 20 3d 20 6f 77 2e 43 72 65 61 74 65 53 68 om.=.ow.CreateSh
0x00879ff8 6f 72 74 63 75 74 28 22 25 73 25 73 22 29 3e 3e ortcut("%s%s")>>
0x0087a008 20 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 6f 6d 2e .m.vbs..echo.om.
0x0087a018 54 61 72 67 65 74 50 61 74 68 20 3d 20 22 25 73 TargetPath.="s
Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a0a8 72 2e 77 6e 72 79 00 00 62 2e 77 6e 72 79 00 00 r.wnry..b.wnry..
0x0087a0b8 61 74 74 72 69 62 20 2b 68 20 2b 73 20 25 43 3a attrib.+h.+s.%C:
0x0087a0c8 5c 25 73 00 24 52 45 43 59 43 4c 45 00 00 00 00 \%.SRECYCLE....
0x0087a0d8 25 00 43 00 3a 00 5c 00 25 00 73 00 00 00 00 00 %C.:.\.%.s....
0x0087a0e8 24 00 52 00 45 00 43 00 59 00 43 00 4c 00 45 00 $.R.E.C.Y.C.L.E.
0x0087a0f8 00 00 00 00 25 00 73 00 5c 00 68 00 69 00 62 00 ....%.s.\.h.i.b.
0x0087a108 73 00 79 00 73 00 25 00 73 00 00 00 20 00 3a 00 s.y.s.%.....
0x0087a118 5c 00 00 00 74 61 73 6b 64 6c 2e 65 78 65 00 00 \..taskdl.exe..
0x0087a128 25 73 0a 00 66 2e 77 6e 72 79 00 00 61 74 00 00 %s..f.wnry..at..
0x0087a138 63 6d 64 2e 65 78 65 20 2f 63 20 73 74 61 72 74 cmd.exe./c.start
0x0087a148 20 2f 62 20 25 73 20 76 73 00 00 00 25 73 20 63 ./b.%s.vs...%s.c
0x0087a158 6f 00 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 o...taskkill.exe
0x0087a168 20 2f 66 20 2f 69 6d 20 6d 79 73 71 6c 64 2e 65 ./f./lm.mysqlde
0x0087a178 78 65 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 xe..taskkill.exe
0x0087a188 20 2f 66 20 2f 69 6d 20 73 71 6c 77 72 69 74 65 ./f./lm.sqlwrite
0x0087a198 72 2e 65 78 65 00 00 00 74 61 73 6b 6b 69 6c 6c r.exe...taskkill
Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a074 75 2e 77 6e 72 79 00 00 25 2e 31 66 20 42 54 43 u.wnry..%.if.BTC
0x0087a084 00 00 00 00 24 25 64 20 77 6f 72 74 68 20 6f 66 ....$.d.worth.of
0x0087a094 20 62 69 74 63 6f 69 6e 00 00 00 00 77 00 62 00 .bitcoin....w.b.
0x0087a0a4 00 00 00 00 72 2e 77 6e 72 79 00 00 62 2e 77 6e ....r.wnry..b.wn
0x0087a0b4 72 79 00 00 61 74 72 69 62 20 2b 68 20 2b 73 ry..attrib.+h.+s
0x0087a0c4 20 25 43 3a 5c 25 73 00 24 52 45 43 59 43 4c 45 %C:\%.SRECYCLE
0x0087a0d4 00 00 00 00 25 00 43 00 3a 00 5c 00 25 00 73 00 ....%.C.:.\.%.s.
0x0087a0e4 00 00 00 00 24 00 52 00 45 00 43 00 59 00 43 00 ....$.R.E.C.Y.C.
0x0087a0f4 4c 00 45 00 00 00 00 00 25 00 73 00 5c 00 68 00 L.E.....%.s.\.h.
0x0087a104 69 00 62 00 73 00 79 00 73 00 25 00 73 00 00 00 t.b.s.y.s.%s...
0x0087a114 20 00 3a 00 5c 00 00 00 74 61 73 6b 64 6c 2e 65 ..:\..taskdl.e
0x0087a124 78 65 00 00 25 73 0a 00 66 2e 77 6e 72 79 00 00 xe..%.s..f.wnry..
0x0087a134 61 74 00 00 63 6d 64 2e 65 78 65 20 2f 63 20 73 at..cmd.exe./c.s
0x0087a144 74 61 72 74 20 2f 62 20 25 73 20 76 73 00 00 00 tart./b.%s.vs...
0x0087a154 25 73 20 63 6f 00 00 00 74 61 73 6b 6b 69 6c 6c %s.co...taskkill
0x0087a164 2e 65 78 65 20 2f 66 20 2f 69 6d 20 6d 79 73 71 .exe./f./lm.mysq

```

Fig. 7. Yarascan Result using Volatility

Yara Rules

The yara rules are a collection of signatures that have been analyzed by malware analysts and are used to identify malware. These rules are specific conditions for each malware and help identify it. Almost every endpoint security company, such as Virustotal, McAfee, and Palo Alto, has used these rules [15].

4.4. Creating a Dataset

After conducting scans and saving the results, the next step is to transform them into a dataset. Utilize the flowchart in Figure 8 to accomplish this.

Steps:-

1. Create Yarascan results of each malware using volatility
2. Remove common signatures that it has by matching them with **standard benign samples**
3. Append these rules into a data frame.
4. Keep a record of benign and malicious samples by creating a **TARGET** column

4.5. Using Artificial Intelligence to Predict Malware

After creating the Dataset, it was time to create the dataset. This involved steps such as

- a. Data Pre-Processing
 - a. Merging data from different Scan Results
 - b. Handling all Duplicated Values
 - c. Scaling Data: - Dividing data values by 255 (16*16 memory samples produced) to scale data between 0 and 1.
 - d. Splitting data into Training and Testing

- b. Exploratory Data Analysis

The Heat map doesn't produce many results. It depicts how features between the range (13,104) show a slight correlation, as shown in Figure 9. This technique could help to reduce dimensions.

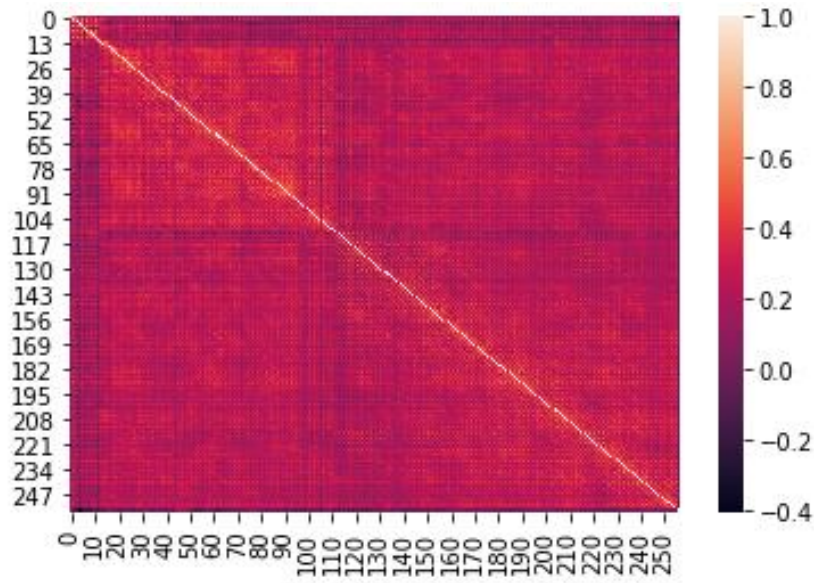


Fig. 9. Heat Map showing data correlation between features.

The count plot of the Target Column shows the number of samples of each class, as shown in Figure 10. On Applying PCA and TSNE to reduce dimensions to visualize each sample, it can be conclusively said that samples tend to form compact clusters, as shown in Figures 11 and 12.

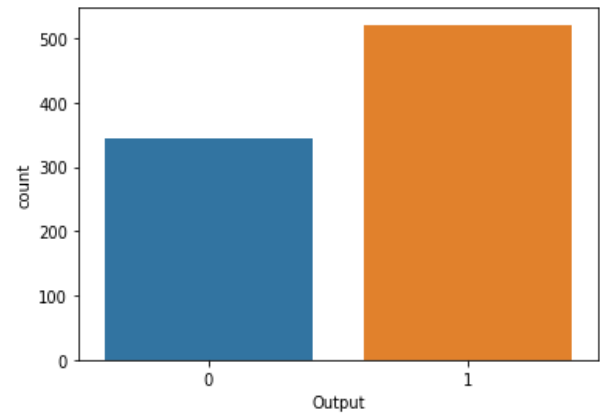


Fig. 10. Distribution of target column

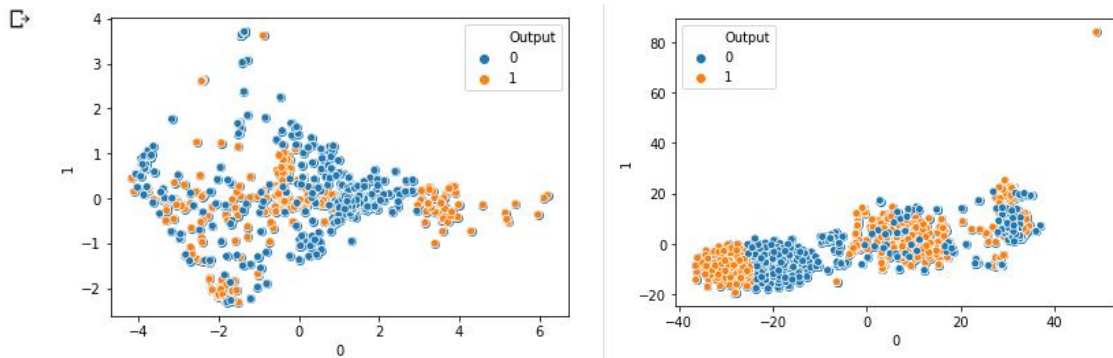


Fig. 11. Reduced Information Distribution of Malware and Benign Samples

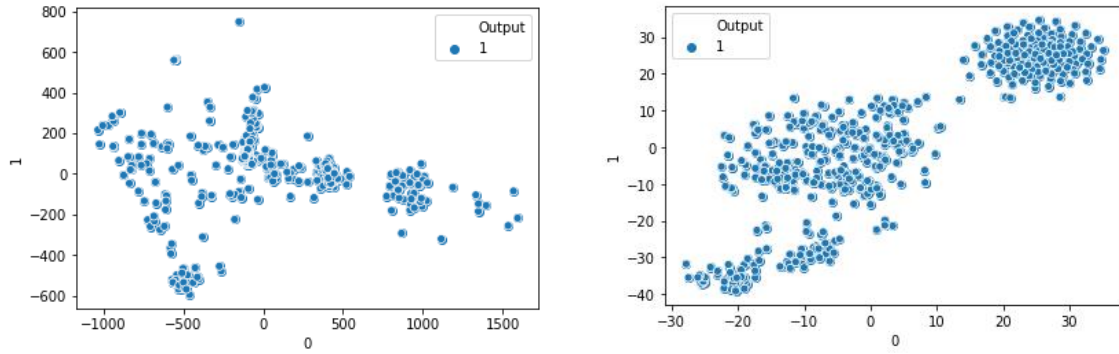


Fig. 12. Reduced Information Distribution of Malware samples only

This helps to identify that classification of such compact clusters needs a highly refined and complex decision boundary/ function.

- c. Applying Machine Learning Algorithms and Deep Learning Algorithms
- On Applying various Machine Learning Algorithms, the metrics accuracy, precision, recall, and F1-Score were computed, which are reported. A detailed discussion of the results is provided in the next chapter.
- Classical supervised learning algorithms seem to perform poorly.

- Ensemble techniques seem to perform better on the dataset.
 - A high Training Score is obtained
- The Deep learning algorithm used currently is ANN (Artificial Neural Network). It consists of dense and dropout layers. The model needs a better model that can remember the previous value, such as an RNN or LSTM, etc.
- The following example of the KNN algorithm proves PCA and TSNE graphs/plots, as shown in Figure 13. As the clusters were compact, and malware and benign samples were close to each other, the value of K had to be low. The score/accuracy should decrease with an increase in K's Nearest neighbors.

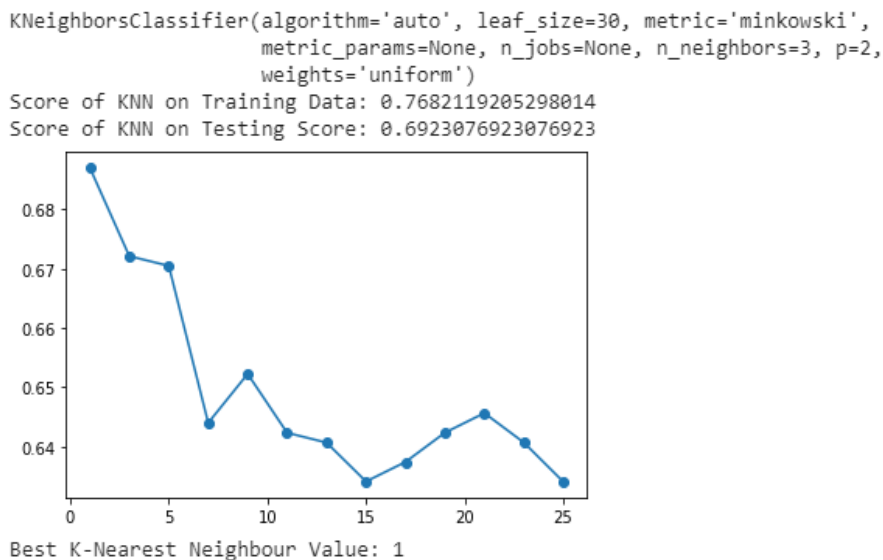


Fig. 13. Using KNN with dataset and mapping score with K using cross-validation

5. Results and Discussion

Here in this section, we have defined the novelty, contributions and results.

5.1. Novelty

This research pioneers an innovative approach to malware detection by introducing a newly developed dataset curated explicitly for this study, marking a significant contribution to the field. The novel methodology employs the extraction of signatures using Volatility for memory forensics and YARA rules for pattern matching, which, for the first time,

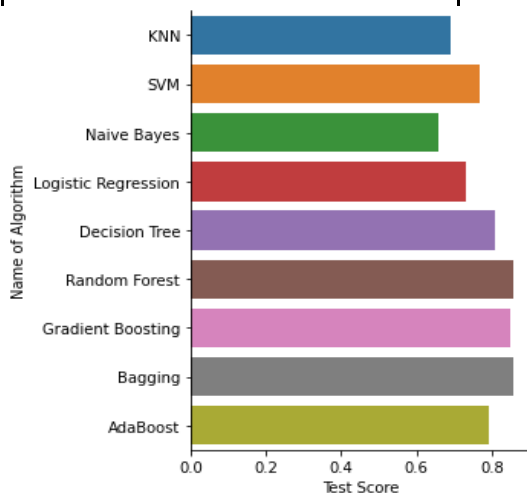
are integrated to train machine learning classifiers. This integration enables the classifiers to effectively recognize active malwares within system memory, setting a new benchmark in cybersecurity research. Our approach demonstrates the feasibility of using memory forensics combined with machine learning for malware detection. It opens new avenues for future research in enhancing malware identification techniques through advanced signature analysis.

5.2. Contributions

This research introduces a groundbreaking approach to malware detection, uniquely combining dynamic memory forensics with advanced machine learning techniques. Analyzing system memory in real-time provides a proactive defense mechanism that identifies and neutralizes active malware based on their signatures and operational patterns. It significantly enhances security measures, particularly in detecting and mitigating hidden trojans that operate covertly. This novel technique contributes a new dimension to cybersecurity defenses. It paves the way for future advancements in integrating memory forensics with machine learning for more sophisticated and effective malware detection systems.

Table 1 Values of different parameters for all classifiers after classification

Name of Classifier	Accuracy	Precision	Recall	F1 Score
KNN Classifier	0.692	0.71	0.69	0.70
SVM Classifier	0.769	0.77	0.77	0.77
Naïve Bayes Classifier	0.657	0.66	0.66	0.66
Logistic Regression	0.730	0.73	0.73	0.73
Decision Tree Classifier	0.80	0.81	0.81	0.81
Random Forest Classifier	0.857	0.86	0.86	0.86
Gradient Boosting Classifier	0.85	0.85	0.85	0.85



5.3. Results

From the Machine Learning Analysis showed that malicious and benign samples formed clusters. Thus, processes exhibiting similar types of behavior in the system have similar memory values. Using this, when we applied machine learning and deep learning to the dataset, we got the following result, as shown in Table 1.

On Applying Machine Learning, to the memory values, the model seems to perform well, as shown in Fig. 14. It has an average training score and can predict the new malwares from the previously learned malwares. The dataset needs to be increased for it to have higher accuracy.

On Applying Machine Learning to the memory values, the model seems to perform well, as shown in Fig. 15. It has a high training score, which it should, as it needs to remember all the values of the malwares given.

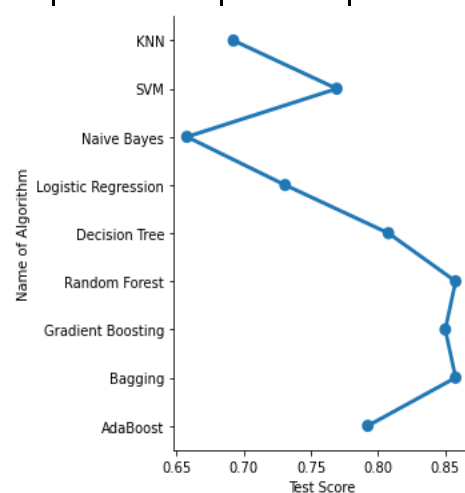
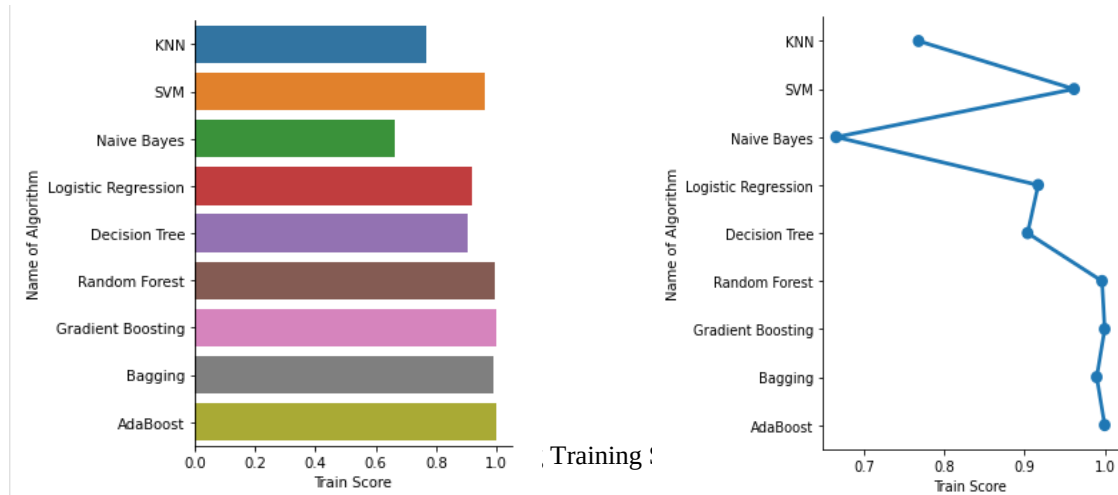


Fig. 14. Comparing Testing Scores of Dataset



6. Conclusion, Limitations, and Future Scope

This study has developed a robust method for the classification of malicious files on end-user computers, aiming to elevate malware detection accuracy and minimize false positives. By integrating additional memory-related features not present in dynamic analysis, this approach surpasses the constraints of a singular execution path, uncovering hidden malware aspects and enhancing detection performance. Utilizing a Random Forest classifier, the research achieved an 89% F-1 score, showcasing the high accuracy in grouping similar process behaviors. This method, independent of operating systems, offers antivirus software a novel technique for identifying malicious activities, marking a significant advancement in cybersecurity practices.

Looking ahead, expanding the YARA rules to incorporate contributions from sources like Alienvault, McAfee, and Palo Alto could broaden the detection scope. Moreover, transitioning this technique into a full-scale antivirus solution necessitates developing a user interface and deepening the malware analysis to understand its impact on memory hexadecimal values, thus refining detection capabilities further.

While this study has demonstrated significant potential in using machine learning for malware detection, particularly with ransomware and benign processes, it acknowledges certain limitations that pave the way for future research. Currently, the classifier's training is restricted to ransomware and non-malicious processes, indicating the necessity to expand its scope to include Trojans, viruses, and other malware varieties for a more comprehensive detection system. Additionally, the initial approach relied on using YARA rules with Volatility due to resource constraints, limiting the analysis to reduced signature datasets rather than the intended full RAM dump analysis.

Future work will extend the range of malware types analyzed and explore the feasibility of training the AI system directly on complete memory snapshots. Moreover, investigating advanced machine learning, deep learning algorithms, and neural network architectures promises to enhance the accuracy and efficiency of malware detection, moving closer to a more robust and versatile cybersecurity solution.

References

- [1] <https://medium.com/bugbountywriteup/intro-to-malware-detection-using-yara-eacab8373cf4>
- [2] Ye, Y., Li, T., Adjeroh, D., & Iyengar, S. S. (2017). A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)*, 50(3), 1-40.
- [3] Sihwail, R., Omar, K., & Ariffin, K. Z. (2018). A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis. *Int. J. Adv. Sci. Eng. Inf. Technol*, 8(4-2), 1662-1671.
- [4] <https://github.com/volatilityfoundation/volatility>
- [5] Choudhary, Sarika. (2013). How Anti-virus Software Works??. *International Journal of Software Engineering and Knowledge Engineering*. 3. 483-484.
- [6] Bhojani, N. (2014). *Malware Analysis*. Malware Analysis, 1-5.
- [7] Tahir, R. (2018). A study on malware and malware detection techniques. *International Journal of Education and Management Engineering*, 8(2), 20.
- [8] Yadav, Deepali & Kumar, Gautam & Kameshwari, D. & Gunjan, Vinit & Kumar, Sheo. (2022). *Malware Techniques and Its Effect: A Survey*. 10.1007/978-981-16-7985-8_127.
- [9] Yusuf, Muchelule & Neyole, Wanjala & Jacob, Misiko & Muchelule, Yusuf & Misiko, Neyole. (2017). *Review of Viruses and Antivirus Patterns*. *Journal of Computer Science and Technology*. 17.

- [10] https://www.blackhat.com/presentations/bh-dc-07/Kendall_McMillan/Presentation/bh-dc-07-Kendall_McMillan.pdf
- [11] Assessing the Effectiveness of YARA Rules for Signature-Based Malware Detection and Classification Adam Lockett
- [12] Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. (2019). Text classification algorithms: A survey. *Information*, 10(4), 150.
- [13] Si, Qin & Xu, Hui & Tong, Ying & Zhou, Yu & Liang, Jian & Cui, Lei & Hao, Zhiyu. (2022). Malware Detection Using Automated Generation of Yara Rules on Dynamic Features. 10.1007/978-3-031-17551-0_21.
- [14] Raghavan, Sriram. (2012). Digital forensic research: Current state of the art. *CSI Transactions on ICT*. 1. 10.1007/s40012-012-0008-7.
- [15] A. R. Javed, W. Ahmed, M. Alazab, Z. Jalil, K. Kifayat and T. R. Gadekallu, "A Comprehensive Survey on Computer Forensics: State-of-the-Art, Tools, Techniques, Challenges, and Future Directions," in *IEEE Access*, vol. 10, pp. 11065-11089, 2022, doi: 10.1109/ACCESS.2022.3142508.
- [16] Hashemi, H.; Hamzeh, A. Visual malware detection using local malicious pattern. *J. Comput. Virol. Hacking Tech*. 2018, 15, 1–14.
- [17] Salehi, Z.; Sami, A.; Ghiasi, M. Using feature generation from API calls for malware detection. *Comput. Fraud Secur*. 2014, 2014, 9–18.
- [18] Cheng, Y.; Fan, W.; Huang, W.; An, J. A Shellcode Detection Method Based on Full Native API Sequence and Support Vector Machine. In *IOP Conference Series: Materials Science and Engineering*; IOP Publishing: Bristol, UK, 2017; Volume 242, pp. 1–7.
- [19] Sun, Z.; Rao, Z.; Chen, J.; Xu, R.; He, D.; Yang, H.; Liu, J. An Opcode Sequences Analysis Method For Unknown Malware Detection. In *Proceedings of the 2019 2nd International Conference on Geoinformatics and Data Analysis, Prague, Czech Republic, 15–17 March 2019*; ACM: New York, NY, USA, 2019.
- [20] Kolosnjaji, B.; Demontis, A.; Biggio, B.; Maiorca, D.; Giacinto, G.; Eckert, C.; Roli, F. Adversarial malware binaries: Evading deep learning for malware detection in executables. In *Proceedings of the 2018 26th European Signal Processing Conference (EUSIPCO)*, Rome, Italy, 3–7 September 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 533–537.
- [21] Egele, M.; Scholte, T.; Kirda, E.; Kruegel, C. A survey on automated dynamic malware-analysis techniques and tools. *ACM Comput. Surv*. 2012, 44, 6.
- [22] Mohaisen, A.; Alrawi, O. Unveiling Zeus Automated Classification of Malware Samples. In *Proceedings of the 22nd International Conference on World Wide Web Companion, Rio de Janeiro, Brazil, 13–17 May 2013*; pp. 829–832.
- [23] Mohaisen, A.; Alrawi, O.; Mohaisen, M. AMAL: High-fidelity, behavior-based automated malware analysis and classification. *Comput. Secur*. 2015, 52, 251–266.
- [24] Liang, G.; Pang, J.; Dai, C. A Behavior-Based Malware Variant Classification Technique. *Int. J. Inf. Educ. Technol*. 2016, 6, 291–295.
- [25] Galal, H.S.; Mahdy, Y.B.; Atiea, M.A. Behavior-based features model for malware detection. *J. Comput. Virol. Hacking Tech*. 2016, 12, 59–67.
- [26] Fan, C.-I.; Hsiao, H.-W.; Chou, C.-H.; Tseng, Y.-F. Malware Detection Systems Based on API Log Data Mining. In *Proceedings of the 2015 IEEE 39th Annual Computer Software and Applications Conference*, Los Alamitos, CA, USA, 10 June 2015; pp. 255–260.
- [27] Ki, Y.; Kim, E.; Kim, H.K. A novel approach to detect malware based on API call sequence analysis. *Int. J. Distrib. Sens. Netw*. 2015, 11, 659101.
- [28] Dai, Y.; Li, H.; Qian, Y.; Lu, X. A malware classification method based on memory dump grayscale image. *Digit. Investig*. 2018, 27, 30–37.
- [29] Dener, M.; Ok, G.; Orman, A. Malware Detection Using Memory Analysis Data in Big Data Environment. *Appl. Sci*. 2022, 12, 8604. <https://doi.org/10.3390/app12178604>
- [30] Machine Learning Analysis of Memory Images for Process Characterization and Malware Detection; S. Lyles, M. DeSantis, M. Gallegos, H. Nyholm, C. Taylor, J. Donaldson, K. Monteith December 16, 2021
- [31] <https://www.malwarebytes.com/malware/>
- [32] <https://antivirus.comodo.com/how-antivirus-software-works.php>
- [33] https://en.wikipedia.org/wiki/Digital_forensics